MAY 19TH, 2023

Open Ended Problem

Microprocessor System

INSTRUCTOR

Ma'am Farah binte Jameel

PRESENTED BY

Hadia Zulfiqar
EE-1513

Khadeeja Khan
EE-1550

Maha Kamran
EE-1556

Narmeen Sabah Siddiqui
CMS: 00000376190



Open Ended Problem

Report on

Line Follower and Obstacle Avoidance Robot

Microprocessor Systems

Ma'am Farah binte Jameel

May 18th , 2023

Semester: 1V- A

Hadia Zulfiqar (EE-1513)

Khadeeja Khan (EE-1550)

Maha Kamran (EE- 1556)

Narmeen Sabah Siddiqui (CMS: 376190)

Contents

Contents	3
The Problem	4
Abstract	4
List of Components:	5
Design And Control	7
Circuit Diagram	9
The code	11

The Problem

Design a project using a microcontroller - Arduino UNO.
Incorporate all the learning you have done in various lab tasks.

Abstract

A line-following robot with obstacle-avoidance capability is an autonomous robot designed to follow a line on a surface while avoiding obstacles in its path. The robot utilizes a combination of sensors, such as infrared or ultrasonic, to detect the line and obstacles in its way. The sensors gather data that is processed by a microcontroller, which then generates the appropriate commands to control the movement of the robot. The robot's ability to navigate around obstacles allows it to function in various environments, making it useful for tasks such as warehouse inventory management or security patrol.

List of Components:

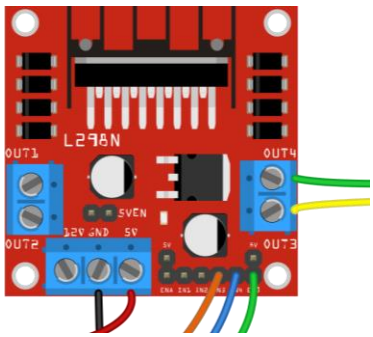
- Arduino Uno MicroController Board



- Servo Motors - TowerPro Micro Servo 9g



- H Bridge Motor Driver



- Ultrasonic Sensor



- Infrared Sensor



- Battery 9V



- DC Motors 12V



- Chassis

- Misc Items: Breadboard, Jumper Wires, Resistors, Leds

Design And Control

The design of a line-following robot consists of the following:

- **Arduino Uno Microcontroller Board**: The microcontroller is responsible for controlling the motors based on the sensor inputs. The Arduino Uno board is a popular choice for LFRs due to its ease of use and low cost.
- **IR sensors:** The LFR detects the line using infrared sensors. The sensors are mounted beneath the robot, facing down toward the ground.
- **H-Bridge Motor Driver:** The motor driver is responsible for controlling the motors that power the LFR's movement. The motor driver receives instructions from the microcontroller and adjusts the speed and direction of the motors accordingly. The H-bridge motor driver is used to control the speed and direction of the motors.
- **Chassis, wheels, and battery holder**: The chassis is a framework for mounting and protecting the components. The wheels allow the robot to move along the line smoothly, and the battery holder keeps the power source secure.
- **DC Motors:** The motors are connected to the wheels and provide the robot with the driving force.

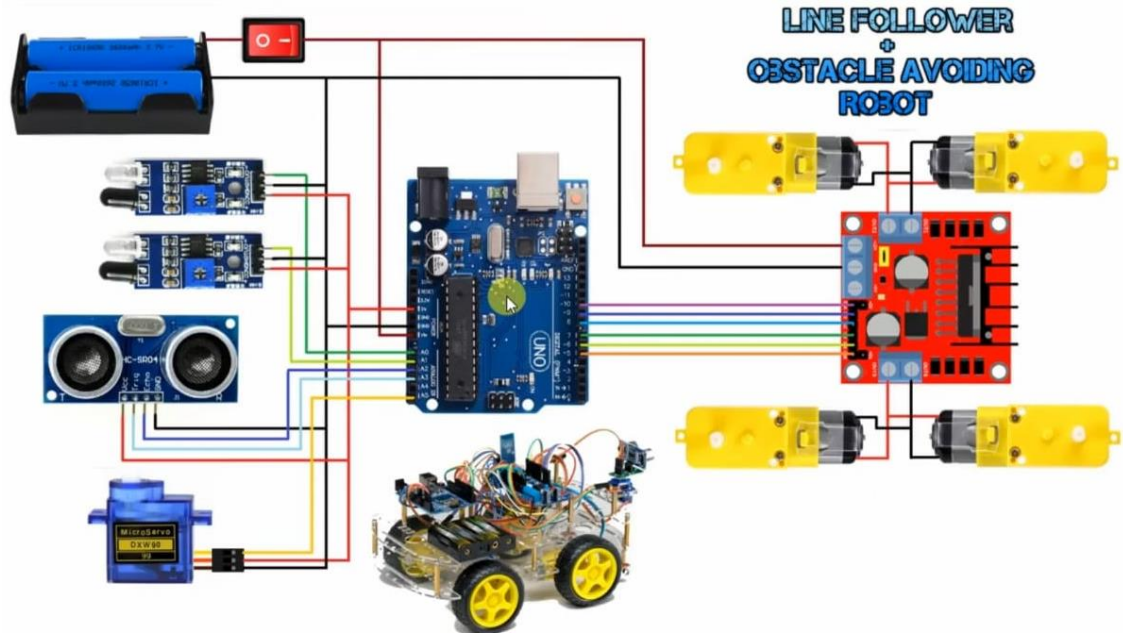
The control of the LFR involves the following steps:

- **Reading sensor inputs:** The Arduino reads the sensor inputs from the infrared sensors. The sensors detect the contrast between the line and the surrounding surface and transmit signals to the microcontroller.
- **Processing sensor inputs:** The Arduino processes the sensor signals and determines the appropriate motor actions to keep the robot on the line. The microcontroller uses a PID (Proportional-Integral-Derivative) algorithm to control the motor speed and direction.
- **Controlling motor actions:** The H-bridge motor driver is used to control the speed and direction of the DC motors. The motor driver receives instructions from the Arduino and adjusts the voltage and current supplied to the motors accordingly.
- **Adjusting motor speed and direction:** The LFR adjusts the speed and direction of the motors based on the sensor inputs. If the robot drifts off the line to the right, the Arduino increases the speed of the left motor and decreases the speed of the right motor to bring the robot back on track.

- **Power source:** The LFR requires a power source to operate. A battery pack or power adapter can be used as the power source, depending on the application.

Overall, designing and controlling the LFR with the Arduino and H-bridge motor driver entails integrating various components and ensuring they all work together to achieve the robot's functionality. The sensors detect the line, the microcontroller uses a PID algorithm to process the sensor signals, and the H-bridge motor driver controls the motors to keep the robot on the line. The chassis, wheels, and battery holder provide a stable foundation for the LFR, allowing it to operate effectively and reliably.

Circuit Diagram



The Line Follower Obstacle Avoidance Robot consists of two infrared sensors and an HC-SR04 ultrasonic sensor attached to it. When the left IR sensor senses the black line, then the robot turns to the left side and if the right IR sensor senses the black line, then the robot turns to the right side. Until both the left and right IR sensors sense the white area, the robot moves forward continuously. If the robot comes across a path where there is another black strip lying perpendicular to the path, then the robot stops at that moment.

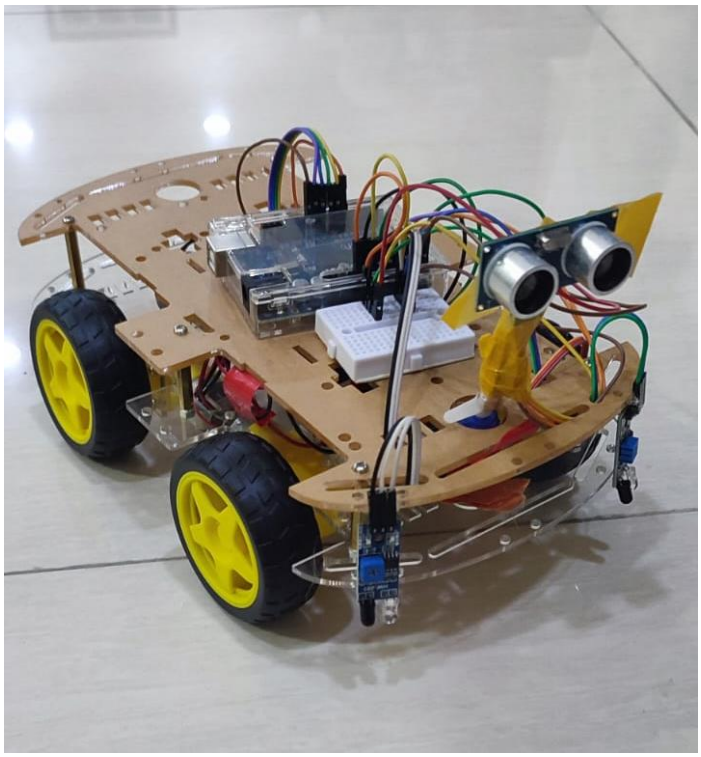
According to the above circuit diagram, we notice that a microcontroller (Atmega328p) is used to communicate with the whole system for processing the robotics functions. First, we need to connect both the VCC and ground pins of the Arduino UNO with a 9V power supply. As we know this is the main source for operating the whole system.

After connecting the above connection, we connect the L293D motor driver shield to the Arduino UNO. Four geared motors are connected to the outer terminal of the motor driver as shown in the circuit diagram. This is a powerful motor driver that can drive four 6V motors easily.

After that, we connected the right infrared sensor's data pin to the A0 pin and the left infrared sensor's data pin to the A1 pin of the Arduino. It helps to detect the black line according to the robot's path and lets it rotate in the correct direction.

Next, we connected an Ultrasonic sensor to detect obstacles in front of its path. Then we need to connect the "Echo" and "Trig" pins to the A4 and A5 pins of the Arduino respectively. It collects the signal by throwing an ultrasound to any obstacles in front of it and then sends this signal by the "Trig" pin.

Thus the Obstacle Avoiding Arduino Line Follower Robot works.



The code

```
//in1: right forward
//in2: right backwards
//in3: left forward
//in4: left backwards

#define enA 10//Enable1 L298 Pin enA
#define in1 9 //Motor1  L298 Pin in1
#define in2 8 //Motor1  L298 Pin in1
#define in3 7 //Motor2  L298 Pin in1
#define in4 6 //Motor2  L298 Pin in1
#define enB 5 //Enable2 L298 Pin enB

#define L_S A0 //ir sensor Left
#define R_S A1 //ir sensor Right

#define echo A2    //Echo pin
#define trigger A3 //Trigger pin

#define servo A5

int Set=15;
int distance_L, distance_F, distance_R;

void setup(){ // put your setup code here, to run once

    Serial.begin(9600); // start serial communication at 9600bps

    pinMode(R_S, INPUT); // declare if sensor as input
    pinMode(L_S, INPUT); // declare ir sensor as input

    pinMode(echo, INPUT );// declare ultrasonic sensor Echo pin as input
    pinMode(trigger, OUTPUT); // declare ultrasonic sensor Trigger pin as
Output

    pinMode(enA, OUTPUT); // declare as output for L298 Pin enA
    pinMode(in1, OUTPUT); // declare as output for L298 Pin in1
    pinMode(in2, OUTPUT); // declare as output for L298 Pin in2
    pinMode(in3, OUTPUT); // declare as output for L298 Pin in3
    pinMode(in4, OUTPUT); // declare as output for L298 Pin in4
    pinMode(enB, OUTPUT); // declare as output for L298 Pin enB

    analogWrite(enA, 169); // Write The Duty Cycle 0 to 255 Enable Pin A for
Motor1 Speed
    analogWrite(enB, 169); // Write The Duty Cycle 0 to 255 Enable Pin B for
Motor2 Speed

    pinMode(servo, OUTPUT);
```

```
    for (int angle = 70; angle <= 140; angle += 5)  {
        servoPulse(servo, angle);  }
    for (int angle = 140; angle >= 0; angle -= 5)  {
        servoPulse(servo, angle);  }

    for (int angle = 0; angle <= 70; angle += 5)  {
        servoPulse(servo, angle);  }

    distance_F = Ultrasonic_read();

    delay(500);
}


void loop(){
    //=============================================
    //     Line Follower and Obstacle Avoiding
    //=============================================

        distance_F = Ultrasonic_read();
        Serial.print("D F=");Serial.println(distance_F);


    //if Right Sensor and Left Sensor are at White color then it will call
forword function
    if((digitalRead(R_S) == 0)&&(digitalRead(L_S) == 0)){
        if(distance_F > Set){
        forword();
        }
        else{
            Check_side();
            }
    }

    //if Right Sensor is Black and Left Sensor is White then it will call turn
Right function
    else if((digitalRead(R_S) == 1)&&(digitalRead(L_S) == 0)){
        turnRight();
        //delay(15);
        }

    //if Right Sensor is White and Left Sensor is Black then it will call turn
Left function
    else if((digitalRead(R_S) == 0)&&(digitalRead(L_S) == 1)){
        turnLeft();
        //delay(15);
        }

    delay(10);
}
```

```
void servoPulse (int pin, int angle){
    int pwm = (angle*11) + 500;      // Convert angle to microseconds
    digitalWrite(pin, HIGH);
    delayMicroseconds(pwm);
    digitalWrite(pin, LOW);
    delay(50); // Refresh cycle of servo
}


//*********************Ultrasonic_read*****************************
long Ultrasonic_read(){
    digitalWrite(trigger, LOW);
    delayMicroseconds(2);
    digitalWrite(trigger, HIGH);
    delayMicroseconds(10);
    long time = pulseIn (echo, HIGH);
    return time / 29 / 2;
}

void compareDistance(){
    if(distance_L > distance_R){
    turnLeft();
    delay(500);
    forword();
    delay(600);
    turnRight();
    delay(500);
    forword();
    delay(600);
    turnRight();
    delay(400);
    }
    else{
    turnRight();
    delay(500);
    forword();
    delay(600);
    turnLeft();
    delay(500);
    forword();
    delay(600);
    turnLeft();
    delay(400);
    }
}

void Check_side(){
    Stop();
    delay(100);
    for (int angle = 70; angle <= 140; angle += 5)  {
    servoPulse(servo, angle);  }
```

```
    delay(300);
    distance_L = Ultrasonic_read();
    Serial.print("D L=");Serial.println(distance_L);
    delay(100);
    for (int angle = 140; angle >= 0; angle -= 5)  {
    servoPulse(servo, angle);  }
    delay(500);
    distance_R = Ultrasonic_read();
    Serial.print("D R=");Serial.println(distance_R);
    delay(100);
    for (int angle = 0; angle <= 70; angle += 5)  {
    servoPulse(servo, angle);  }
    delay(300);
    compareDistance();
}

void forword(){  //forword
digitalWrite(in1, HIGH); //right Motor forword Pin
digitalWrite(in2, LOW); //right Motor backword Pin
digitalWrite(in3, HIGH); //left Motor forword Pin
digitalWrite(in4, LOW); //left Motor backword Pin
}

void backword(){ //backword
digitalWrite(in1, LOW);
digitalWrite(in2, HIGH);
digitalWrite(in3, LOW);
digitalWrite(in4, HIGH);
}

void turnRight(){ //turnRight
digitalWrite(in1, LOW);
digitalWrite(in2, HIGH);
digitalWrite(in3, HIGH);
digitalWrite(in4, LOW);
}

void turnLeft(){ //turnLeft
digitalWrite(in1, HIGH);
digitalWrite(in2, LOW);
digitalWrite(in3, LOW);
digitalWrite(in4, HIGH);
}

void Stop(){ //stop
digitalWrite(in1, LOW);
digitalWrite(in2, LOW);
digitalWrite(in3, LOW);
digitalWrite(in4, LOW);
}
```

Applications

LFR (Line Following Robot) and Obstacle Avoidance Robots are two popular types of robots used in various fields.

Here are some of the applications of LFR and Obstacle Avoidance Robot:

- Industrial automation: LFRs and Obstacle Avoidance Robots are used in various manufacturing industries for carrying out tasks like material handling, packaging, and transportation of goods from one place to another.

- Surveillance: LFRs and Obstacle Avoidance Robots can be used for monitoring and surveillance purposes, especially in areas that are difficult for humans to access or are hazardous.

- Agriculture: LFRs and Obstacle Avoidance Robots can be used in agriculture for tasks such as planting, harvesting, and monitoring crop growth.

- Home automation: LFRs and Obstacle Avoidance Robots can be used for home automation tasks like cleaning, cooking, and managing home appliances.

- Education: LFRs and Obstacle Avoidance Robots can be used in educational settings to teach students about robotics, programming, and automation.

- Medical applications: Obstacle Avoidance Robots can be used in healthcare settings to help people with disabilities or elderly people with mobility issues to move around and perform daily activities.

- Military applications: LFRs and Obstacle Avoidance Robots can be used in military applications for tasks like surveillance, bomb disposal, and reconnaissance missions.

Overall, LFRs and Obstacle Avoidance Robots are versatile and can be used in various fields to automate tasks and make them more efficient.